

BHTTP/1: a binary framing for HTTP

Specification, version 1

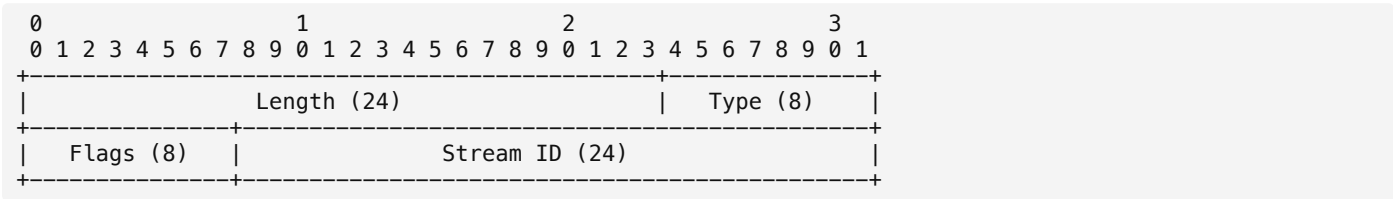
MUST, MUST NOT, SHOULD and MAY are used as in RFC 2119. Integers are unsigned and big-endian. Sizes are in bytes.

1. Connection

BHTTP/1 runs over one TCP connection (default port 9000) with no preface. The client sends its requests one after another and the server answers each, in order, on the same connection, which stays open until the client closes it. A client MUST NOT open a second connection during one run (one set of requests). If the server closes the connection, the client reports the remaining requests as failed rather than reconnecting. A client SHOULD wait for the end of a response before sending the next request. A server MUST still accept requests sent earlier (pipelining) and answer them in order. A server MAY close a connection that is idle, or that takes too long to deliver a request; bserve allows 30 s for each.

2. Frame header

Every frame is an 8-byte header followed by Length bytes of payload.



- Length: payload size, not counting the header (0 to 16 777 215).
- Type: 0x00 DATA, 0x01 HEADERS. Any other value is an unknown type (§3).
- Flags: 0x01 END_STREAM marks the last frame of a request or response. Senders set the other bits to 0 and receivers ignore them.
- Stream ID: the client numbers its requests on a connection, starting at 1. Each request MUST use a higher ID than the one before (gaps are allowed), and the response carries the same ID. Stream 0 stands for the connection itself: in v1 it carries only unknown frames and the connection-error response of §6. After using ID 16 777 215 the client MUST close the connection.

3. Unknown frame types

A receiver that reads a frame of unknown type MUST skip it: read and discard exactly Length payload bytes, then go on to the next frame. It MUST NOT reply to it, close the connection over it, or let it change any request state, whatever its stream ID or flags (END_STREAM on an unknown frame ends nothing). This holds everywhere in the frame sequence, including between the frames of one request or response. Types 0xF0–0xFF will never be assigned. Senders may use them to check that peers skip properly (bcurl --grease sends 0xFA). Skipping works in small chunks, so it costs time, not memory, and §1's timeouts bound the time. HTTP/2 has the same rule (RFC 9113 §4.1, §5.5).

4. Messages

A request is one HEADERS frame and then zero or more DATA frames on the same stream, with END_STREAM on the last frame. A response has the same shape. A message with no body is a single HEADERS frame with END_STREAM set. The header block MUST fit in one HEADERS frame. Receivers MUST accept blocks of up to 65 536 bytes holding any number of fields. Senders SHOULD keep DATA frames to 16 384 bytes or less; receivers MUST accept any Length. A body ends at END_STREAM. content-length is for information only and receivers do not check it: a response to HEAD carries the file's length but no DATA.

5. Header block

A HEADERS payload is a sequence of fields, with no count and no terminator; the frame's Length bounds it. In the grammar, tag is one byte and len is one or two bytes.

field	=	tag [name] value	; name present only when tag = 0
name	=	len bytes	value = len bytes
len	=	0xxxxxxx	; 0 to 127
		1xxxxxxx xxxxxxxx	; 0 to 32 767 (15 bits, big-endian)

Tag 0 means the name follows as a literal. Tags 1 to 10 select a name from the table below, which lists every name bserve and bcurl send. Tags 11 to 255 are not defined in v1.

tag	name	tag	name	tag	name	tag	name	tag	name
1	:method	3	:status	5	user-agent	7	server	9	content-type
2	:path	4	host	6	accept	8	date	10	content-length

- Names are 1 to 32 767 bytes of visible ASCII (0x21–0x7E) with no capital letters. A name starting with : is a pseudo-header and may only be sent through its tag. Senders use the tag for any name in the table. Receivers also accept the other table names as literals.

- Values are 0 to 32 767 bytes and may hold any byte except NUL, CR and LF. Senders SHOULD use the one-byte len when it fits; receivers accept both forms.
- Pseudo-headers come before all other fields. Other names may repeat.
- A request has exactly one :method and one :path, and no :status. The path starts with / and is at most 1024 bytes. host SHOULD be sent.
- A response has exactly one :status, three ASCII digits from 100 to 599, and no :method or :path. Every v1 response is final; there are no interim (1xx) responses before it.
- A block that breaks any of these rules, or uses tags 11 to 255, is malformed.

6. Server

To find the file, the server cuts :path at the first ? or #, decodes %XX escapes, and refuses any . . segment. It then resolves the result under its document root, following symbolic links, and checks that it still lies inside the root. A path that ends in /, or that names a directory, means that directory's index.html. A found file gets 200 with content-type, content-length, server and date, then the file as DATA. A HEAD request gets the same headers and no DATA. Errors get a short text/plain body.

status	when
400	malformed header block (§5), header block over 65 536 bytes, bad % escape or %00 in the path
403	. . segment, path resolving outside the root, or a file the server may not read
404	no regular file at that path
405	a method other than GET or HEAD

These are stream errors: the response goes on the request's stream and the connection stays open. Any request body is read and discarded first. A connection error means the frame sequence cannot be trusted any more. The cases are: HEADERS on stream 0 or with an ID not higher than the last; DATA with no request open or on a different stream; and a new HEADERS before the open request's END_STREAM. The server sends a 400 response on stream 0 with the reason in its body, then closes. If a frame is cut off by EOF the peer has gone, so the server just closes.

7. Client

The client treats the following as protocol errors, closes the connection and exits with 3: a response on a stream other than the current request's, DATA before HEADERS, a second HEADERS in one response, a malformed response header block, a frame cut off by EOF, or 30 s with no data. A 400 on stream 0 is a connection error report, which bcurl prints to stderr before exiting 3. bcurl writes bodies to stdout. With -v it hexdumps every frame it sends (>) and receives (<) to stderr, unknown ones included. Exit codes: 0 if every status was below 400, 4 if any was 4xx, 5 if any was 5xx, 3 for a protocol, timeout or output error, 2 if it cannot connect, 1 for bad arguments.

8. Design notes

HTTP/2's header is Length 24, Type 8, Flags 8, then a reserved bit and a 31-bit Stream ID: 9 bytes. Its Length allows frames up to 16 MiB, but the default maximum is 16 KiB so that, with many streams sharing one connection, a single large frame cannot hold up the others for long. Peers that want bigger frames raise the limit with SETTINGS. A type byte and a flags byte cover its ten frame types and their options, with room to spare. Many streams are open at once and both ends can start them, so the ID space is large. Client streams are odd and server streams are even, which splits the values rather than costing a bit. The top bit is simply reserved with no meaning (RFC 9113 §4.1); it is usually traced back to SPDY, where the first bit of a frame marked control frames.

BHTTP/1 uses 24 / 8 / 8 / 24, which makes 8 bytes.

- Length 24. v1's own limits (16 KiB DATA, 64 KiB header blocks) almost fit 16 bits. The extra byte is headroom for a v2 that wants larger frames. The width cannot be changed later without breaking §3, because skipping depends on every version reading Length the same way.
- Type 8 and Flags 8. v1 uses two types and one flag. The rest is space a v2 can use, and §3 makes using it safe.
- Stream ID 24. v1 has one request in progress at a time and no server-initiated streams, so it needs no odd/even split and no reserved bit. The ID still matches each response to its request and lets a receiver notice a stale or misplaced frame. 16.7 million requests is more than one connection will see, and a v2 can add multiplexing without a new header.

HPACK has a 61-entry static table of name-value pairs, a dynamic table, Huffman coding and prefix-coded integers. BHTTP/1 takes two of its ideas: names from a fixed table (as in HPACK's "literal with indexed name") and length-prefixed literals for everything else. Values are always literal, and the length prefix is a simpler one- or two-byte form. A typical request is 57 bytes on the wire (8-byte header plus a 49-byte block); the same request as HTTP/1.1 text is 85 bytes.

To grow, a v2 peer would first send a new frame type, say SETTINGS, listing what it supports. A v1 peer skips it and never acknowledges it, so a v2 peer that gets its first response with no acknowledgement knows to stay with v1. No timer is needed.